

BAB 1

PENDAHULUAN

1.1 Latar Belakang

1.2 Rumusan Masalah

Rumusan masalah dari tugas akhir ini adalah sebagai berikut:

- Bagaimana cara membangun perangkat lunak permainan *Colored Queens*?
- Bagaimana membangun solusi permainan *Colored Queens* menggunakan teknik *Backtracking* dan *Particle Swarm Optimization* (PSO)?
- Bagaimana membangun solver untuk permainan colored queen yang mengimplementasikan *Backtracking* dan PSO yang dapat diintegrasikan dengan perangkat lunak yang dibangun?
- Bagaimana kinerja dari solver yang dibangun dalam mencari solusi permainan *Colored Queens*?

1.3 Tujuan

Selanjutnya, tujuan dari tugas akhir ini dapat dirumuskan sebagai berikut:

- Membangun perangkat lunak permainan *Colored Queens*.
- Mempelajari cara membangun solusi permainan Colored Queen menggunakan teknik *Backtracking* dan *Particle Swarm Optimization*.
- Membangun solver untuk permainan colored queen yang mengimplementasikan *Backtracking* dan PSO yang dapat diintegrasikan dengan perangkat lunak yang dibangun.
- Melakukan pengujian untuk mengukur kinerja dari solver yang dibangun dalam mencari solusi permainan *Colored Queens*

1.4 Batasan Masalah

Untuk mempermudah pembuatan template ini, tentu ada hal-hal yang harus dibatasi, misalnya saja bahwa template ini bukan berupa style L^AT_EX pada umumnya (dengan alasannya karena belum mampu jika diminta membuat seperti itu)

1.5 Metodologi

Tentunya akan diisi dengan metodologi yang serius sehingga templatnya terkesan lebih serius.

1 Morbi luctus, wisi viverra faucibus pretium, nibh est placerat odio, nec commodo wisi enim
2 eget quam. Quisque libero justo, consectetur a, feugiat vitae, porttitor eu, libero. Suspendisse
3 sed mauris vitae elit sollicitudin malesuada. Maecenas ultricies eros sit amet ante. Ut venenatis
4 velit. Maecenas sed mi eget dui varius euismod. Phasellus aliquet volutpat odio. Vestibulum ante
5 ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Pellentesque sit amet pede
6 ac sem eleifend consectetur. Nullam elementum, urna vel imperdiet sodales, elit ipsum pharetra
7 ligula, ac pretium ante justo a nulla. Curabitur tristique arcu eu metus. Vestibulum lectus. Proin
8 mauris. Proin eu nunc eu urna hendrerit faucibus. Aliquam auctor, pede consequat laoreet varius,
9 eros tellus scelerisque quam, pellentesque hendrerit ipsum dolor sed augue. Nulla nec lacus.

10 1.6 Sistematika Pembahasan

11 Rencananya Bab 2 akan berisi petunjuk penggunaan template dan dasar-dasar L^AT_EX. Mungkin
12 bab 3,4,5 dapt diisi oleh ketiga jurusan, misalnya peraturan dasar skripsi atau pedoman penulisan,
13 tentu jika berkenan. Bab 6 akan diisi dengan kesimpulan, bahwa membuat template ini ternyata
14 sungguh menghabiskan banyak waktu.

BAB 2

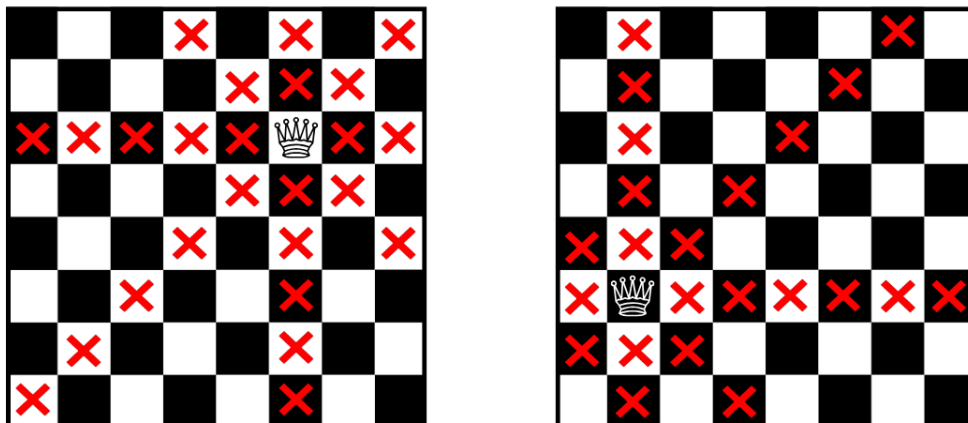
LANDASAN TEORI

2.1 Permasalahan N-Queens

2.1.1 Definisi dan Representasi

Permasalahan N-Queens merupakan salah satu permasalahan klasik dalam bidang ilmu komputer dan matematika diskrit, dengan sejarah panjang yang didokumentasikan secara komprehensif dalam survei yang dilakukan oleh Bell dan Stevens [1]. Permasalahan ini pertama kali diperkenalkan oleh Max Bezzel pada tahun 1848 dalam majalah catur Jerman *Berliner Schachzeitung*. Formulasi awal yang diajukan Bezzel hanya membahas kasus papan berukuran 8×8 , dan baru pada tahun 1850 Franz Nauck mempublikasikan seluruh 92 solusi untuk kasus tersebut sekaligus memperumum permasalahan ke papan berukuran $n \times n$.

Pada formulasi umumnya, diberikan sebuah papan catur berukuran $n \times n$ dan tugasnya adalah menempatkan n buah bidak menteri sedemikian rupa sehingga tidak ada dua menteri yang saling menyerang. Sebagaimana dalam aturan permainan catur, sebuah menteri dapat bergerak secara bebas dalam arah horizontal, vertikal, ataupun diagonal dengan jarak tak terbatas, sehingga setiap penempatan menteri harus mempertimbangkan seluruh arah serangan tersebut seperti yang tertera pada Gambar 2.1.



Gambar 2.1: Contoh aturan penyerangan bidak menteri pada permainan catur. Silang merah menandakan kotak yang terserang oleh bidak menteri.

Dalam praktiknya, papan dapat direpresentasikan menggunakan beberapa cara. Representasi paling intuitif adalah matriks biner berukuran $n \times n$, di mana nilai 1 menandakan keberadaan menteri dan nilai 0 menandakan kotak kosong. Namun, representasi ini tidak efisien dari sudut pandang komputasi karena memungkinkan konfigurasi yang secara trivial tidak valid, seperti dua menteri pada baris yang sama. Oleh sebab itu, representasi yang lebih umum digunakan dalam literatur adalah representasi berbasis permutasi, yaitu sebuah array satu dimensi $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ dengan π_i menyatakan kolom tempat menteri pada baris ke- i ditempatkan [2]. Karena π merupakan permutasi dari himpunan $\{1, 2, \dots, n\}$, representasi ini secara implisit menjamin tidak adanya konflik baris maupun konflik kolom, sehingga ruang pencarian yang perlu dieksplorasi berkurang dari n^n menjadi $n!$ konfigurasi.

Dengan demikian, permasalahan N-Queens pada dasarnya dapat dirumuskan sebagai pencarian permutasi π yang memenuhi kondisi bahwa tidak ada dua menteri yang berbagi diagonal yang sama. Secara formal, π merupakan solusi valid jika dan hanya jika untuk setiap pasangan indeks $i \neq j$ berlaku $|i - j| \neq |\pi_i - \pi_j|$ [2]. Formulasi ringkas ini menjadi titik berangkat bagi pemodelan permasalahan dalam kerangka kerja yang lebih formal, yang akan dibahas pada Bagian 2.3.

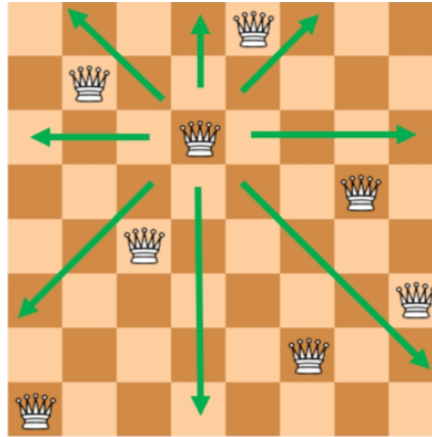
2.1.2 Aturan Konflik Queen

Aturan pergerakan bidak menteri pada permainan catur memunculkan tiga jenis konflik yang harus dihindari secara simultan dalam permasalahan N-Queens. Ketiga jenis konflik tersebut bersesuaian dengan tiga arah pergerakan menteri, yaitu horizontal, vertikal, dan diagonal [1].

Konflik horizontal terjadi apabila terdapat dua menteri yang ditempatkan pada baris yang sama. Mengingat sebuah menteri dapat bergerak bebas sepanjang barisnya, kedua menteri tersebut akan saling menyerang tanpa memandang jarak di antara keduanya. Secara analog, konflik vertikal terjadi apabila dua menteri menempati kolom yang sama. Kedua jenis konflik ini bersifat langsung dan mudah diidentifikasi karena hanya melibatkan kesetaraan indeks baris atau kolom.

Konflik diagonal memerlukan analisis yang sedikit lebih cermat. Sebuah menteri dapat menyerang sepanjang dua jenis diagonal, yaitu diagonal utama yang membentang dari kiri atas ke kanan bawah, serta diagonal anti yang membentang dari kanan atas ke kiri bawah. Untuk dua menteri yang ditempatkan pada posisi (i, π_i) dan (j, π_j) , keduanya berada pada diagonal yang sama apabila selisih baris dan selisih kolomnya memiliki nilai absolut yang setara, yaitu $|i - j| = |\pi_i - \pi_j|$. Karakterisasi aritmatika ini memanfaatkan sifat geometris papan: seluruh kotak pada diagonal utama memiliki nilai $i - \pi_i$ yang konstan, sedangkan seluruh kotak pada diagonal anti memiliki nilai $i + \pi_i$ yang konstan [3].

Berdasarkan ketiga jenis konflik di atas, permasalahan N-Queens dapat dirumuskan secara formal sebagai pencarian penempatan n menteri pada papan $n \times n$ yang memenuhi tiga kendala berikut secara bersamaan: (1) tidak ada dua menteri yang berbagi baris yang sama; (2) tidak ada dua menteri yang berbagi kolom yang sama; dan (3) tidak ada dua menteri yang berbagi diagonal yang sama [3]. Sebagaimana telah dibahas pada Bagian 2.1.1, penggunaan representasi permutasi $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ secara implisit menghilangkan dua kendala pertama, sehingga hanya kendala diagonal yang perlu diperiksa secara eksplisit selama proses pencarian solusi.



Gambar 2.2: Contoh solusi valid permasalahan 8-Queens. Panah hijau menandakan jangkauan serangan salah satu menteri; tidak ada menteri lain yang berada pada baris, kolom, ataupun diagonal yang sama.

Interaksi antarkendala inilah yang membuat permasalahan N-Queens menarik dari sudut pandang komputasi. Pelanggaran terhadap salah satu kendala mengakibatkan konfigurasi menjadi tidak valid, dan kendala-kendala tersebut tidak dapat diperiksa secara terpisah karena keputusan penempatan satu menteri secara langsung memengaruhi domain posisi valid bagi menteri-menteri lainnya. Sifat saling-bergantung antarkendala ini menjadi dasar bagi pemodelan permasalahan menggunakan kerangka kerja *Constraint Satisfaction Problem* yang akan dibahas pada Bagian 2.3.

2.1.3 Kompleksitas dan Relevansi

Dari perspektif komputasi, permasalahan N-Queens menarik karena memiliki struktur yang sederhana namun ruang solusinya sangat besar dan tumbuh secara eksponensial. Apabila tidak ada asumsi tambahan yang digunakan, setiap menteri memiliki n^2 kemungkinan posisi pada papan, menghasilkan ruang pencarian awal sebesar $(n^2)^n$. Penggunaan representasi permutasi sebagaimana dijelaskan pada Bagian 2.1.1 mempersempit ruang ini menjadi $n!$, namun pertumbuhannya tetap superpolynomial. Sebagai gambaran, untuk $n = 14$ terdapat $14! \approx 8,7 \times 10^{10}$ kemungkinan permutasi yang harus dievaluasi, jumlah yang jauh melampaui kemampuan komputasi *brute-force* konvensional pada perangkat keras umum.

Meskipun ruang pencarian tumbuh secara eksponensial, perlu ditekankan bahwa permasalahan N-Queens dalam formulasi dasarnya bukanlah permasalahan yang sulit secara teoretis. Hoffman, Loessi, dan Moore [4] sebagaimana dirangkum dalam survei Bell dan Stevens [1] menunjukkan bahwa terdapat konstruksi aritmatika eksplisit yang dapat menghasilkan satu solusi N-Queens untuk setiap $n \geq 4$ dalam waktu linier $O(n)$ tanpa perlu melakukan pencarian sama sekali. Dengan demikian, persoalan keputusan “apakah solusi N-Queens ada untuk papan $n \times n$ ” dapat dijawab dalam waktu konstan: solusinya selalu ada kecuali untuk $n = 2$ dan $n = 3$.

Karakteristik kompleksitas N-Queens telah menjadi topik diskusi yang panjang dalam komunitas *Artificial Intelligence*, khususnya terkait kelayakannya sebagai *benchmark* algoritma pencarian. Gent, Jefferson, dan Nightingale [5] memberikan analisis komprehensif terhadap persoalan ini dan menunjukkan bahwa permasalahan N-Queens dalam bentuk dasarnya tidak ideal sebagai tolok ukur kesulitan algoritma, mengingat solusinya dapat dikonstruksi secara langsung tanpa

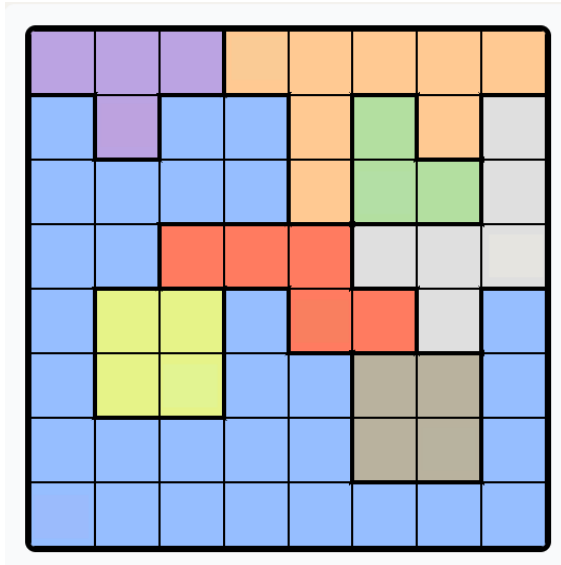
1 proses pencarian. Sebaliknya, mereka membuktikan bahwa varian *N-Queens Completion*, yaitu
2 permasalahan menyelesaikan konfigurasi parsial yang telah diberikan sebelumnya, tergolong NP-
3 Complete dan #P-Complete. Hasil ini mengindikasikan bahwa varian-varian N-Queens dengan
4 kendala tambahan atau struktur papan ireguler memiliki kompleksitas teoritis yang setara dengan
5 permasalahan-permasalahan sulit klasik dalam ilmu komputer.

6 Permasalahan *Colored Queens* yang menjadi fokus tugas akhir ini memiliki struktur kendala
7 yang jauh lebih kompleks dibandingkan N-Queens klasik, termasuk pembagian sektor warna dan
8 kendala *adjacency* yang tidak dapat diselesaikan dengan konstruksi aritmatika sederhana. Dengan
9 kata lain, meskipun teknik konstruktif tersedia untuk N-Queens dasar, teknik tersebut tidak dapat
10 diperumum ke varian-varian dengan kendala ireguler. Hal inilah yang menjadikan pendekatan
11 algoritmik berbasis pencarian, baik yang bersifat sistematis maupun *metaheuristic*, tetap relevan
12 dan menjadi fokus pembahasan pada bab-bab selanjutnya.

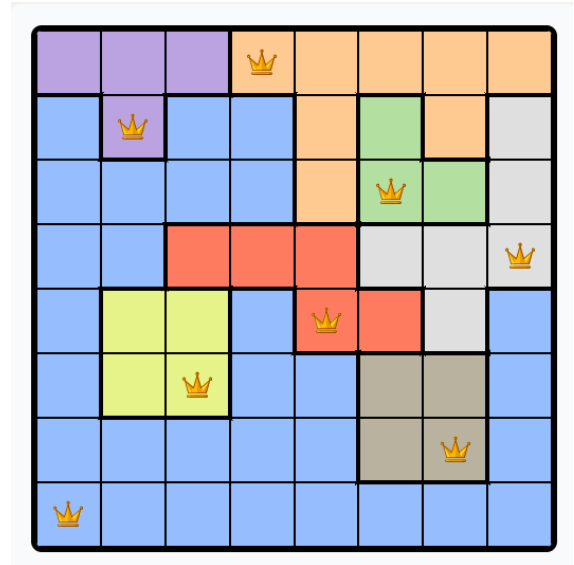
13 2.2 Colored Queens Problem

14 2.2.1 Definisi dan Aturan Permainan

15 Permasalahan *Colored Queens* merupakan sebuah *puzzle* logika berbasis papan yang dipopulerkan
16 melalui permainan *Queens* pada platform LinkedIn [6]. Secara struktural, permasalahan ini memiliki
17 kesamaan yang kuat dengan jenis *puzzle* penempatan objek yang dikenal sebagai *Star Battle*, yang
18 pertama kali dirancang oleh Hans Eendebak untuk *World Puzzle Championship* tahun 2003 [7].
19 Dalam varian yang dibahas pada tugas akhir ini, permainan menggunakan papan berukuran $n \times n$
20 yang dibagi menjadi n buah sektor berwarna (*colored regions*) dengan bentuk yang umumnya tidak
21 beraturan (*irregular*). Setiap sektor warna membentuk satu wilayah yang terhubung (*connected*
22 *region*) pada papan, artinya seluruh sel dalam satu sektor dapat dijangkau dari sel lainnya melalui
23 tetangga horizontal atau vertikal tanpa melewati sel dari sektor lain. Contoh papan permainan
24 beserta solusinya diperlihatkan pada Gambar 2.3.



(a) Papan awal



(b) Papan yang telah diselesaikan

Gambar 2.3: Contoh papan permainan *Colored Queens* berukuran 8×8 dengan 8 sektor warna. Setiap sektor membentuk wilayah terhubung, dan solusi menempatkan tepat satu menteri per baris, kolom, dan sektor warna tanpa ada dua menteri yang bertetangga.

Tujuan permainan adalah menempatkan tepat n buah bidak menteri pada papan sedemikian rupa sehingga seluruh aturan berikut terpenuhi secara bersamaan:

1. Setiap baris memuat tepat satu menteri.
2. Setiap kolom memuat tepat satu menteri.
3. Setiap sektor warna memuat tepat satu menteri.
4. Tidak ada dua menteri yang menempati sel-sel yang bertetangga (*adjacent*), termasuk tetangga diagonal. Secara formal, dua sel (r_1, c_1) dan (r_2, c_2) dikatakan bertetangga apabila $|r_1 - r_2| \leq 1$ dan $|c_1 - c_2| \leq 1$.

Perlu dicatat bahwa, berbeda dengan permasalahan N-Queens klasik yang dibahas pada Bagian 2.1, bidak menteri dalam *Colored Queens* **tidak** menyerang sepanjang diagonal. Serangan hanya berlaku dalam arah horizontal dan vertikal, sehingga kendala diagonal digantikan oleh kendala *adjacency* yang bersifat lokal (hanya memengaruhi delapan sel di sekeliling menteri). Perbedaan ini mengubah karakteristik ruang pencarian secara fundamental, karena kendala *adjacency* tidak dapat direduksi menjadi perbandingan aritmatika sederhana sebagaimana kendala diagonal pada N-Queens.

Selain itu, keberadaan sektor warna yang berbentuk ireguler menambah dimensi kendala yang tidak dimiliki oleh N-Queens. Pada N-Queens, satu-satunya entitas pembatas adalah baris, kolom, dan diagonal yang semuanya memiliki struktur geometris teratur. Pada *Colored Queens*, sektor warna memiliki bentuk yang bervariasi antarpapan dan tidak mengikuti pola geometris yang dapat diprediksi, sehingga setiap instansi permainan memerlukan analisis kendala yang berbeda.

2.2.2 Kompleksitas dan Struktur Masalah

Permasalahan *Colored Queens* memiliki struktur kendala berlapis (*multi-layered constraints*) yang membedakannya secara fundamental dari N-Queens klasik. Sebagaimana diformulasikan oleh

Mata Ali dan Mencia [8], permainan ini merupakan varian dari N-Queens yang melonggarkan kendala diagonal namun menambahkan kendala sektor warna dan *adjacency*. Kombinasi kendala-kendala tersebut menghasilkan interaksi yang lebih kompleks dibandingkan N-Queens, karena setiap keputusan penempatan menteri secara simultan memengaruhi tiga domain pembatas yang berbeda: baris/kolom, sektor warna, dan lingkungan tetangga.

Lapisan pertama adalah kendala baris dan kolom, yang secara struktural identik dengan N-Queens dan membatasi setiap baris serta kolom untuk memuat tepat satu menteri. Lapisan kedua adalah kendala sektor warna, yang mengharuskan setiap sektor memiliki tepat satu menteri. Berbeda dengan kendala baris dan kolom yang bersifat reguler dan simetris, sektor warna memiliki bentuk ireguler yang bervariasi antarpapan. Implikasinya, ukuran domain untuk setiap sektor (jumlah sel yang tersedia) tidak seragam, dan beberapa sektor mungkin memiliki domain yang jauh lebih kecil dibandingkan sektor lainnya. Hal ini menciptakan ketidakseimbangan dalam proses pencarian: penempatan menteri pada sektor berukuran kecil memiliki dampak propagasi yang berbeda dibandingkan penempatan pada sektor berukuran besar.

Lapisan ketiga adalah kendala *adjacency*, yang melarang dua menteri menempati sel-sel yang bertetangga. Kendala ini bersifat lokal dan simetris, namun efeknya bersifat global karena penempatan satu menteri secara langsung mengeliminasi hingga delapan sel tambahan dari domain menteri-menteri lainnya. Pada papan berukuran kecil ($n \leq 8$), eliminasi ini sangat signifikan secara proporsional terhadap total sel yang tersedia.

Interaksi antara ketiga lapisan kendala tersebut menciptakan beberapa tantangan komputasi. Pertama, tidak ada representasi permutasi sederhana sebagaimana pada N-Queens, karena variabel terkait erat dengan sektor warna yang bentuknya tidak teratur. Kedua, kendala *adjacency* tidak dapat diekspresikan sebagai ketidaksamaan aritmatika global (seperti $|i - j| \neq |\pi_i - \pi_j|$ pada N-Queens), melainkan harus diperiksa secara lokal untuk setiap pasangan sel. Ketiga, bentuk ireguler sektor warna menyebabkan *constraint graph* yang dihasilkan tidak memiliki struktur simetris yang dapat dieksploitasi oleh teknik konstruktif.

Dari sudut pandang kompleksitas, meskipun belum terdapat pembuktian formal bahwa *Colored Queens* termasuk dalam kelas NP-Complete, struktur permasalahannya memiliki kemiripan dengan *N-Queens Completion* yang telah terbukti NP-Complete [5]. Keduanya melibatkan kendala tambahan yang merusak keteraturan geometris N-Queens dasar, sehingga teknik konstruktif tidak dapat diterapkan. Kompleksitas inilah yang menjadikan permasalahan ini menarik untuk diselesaikan secara algoritmik menggunakan pendekatan berbasis pencarian.

2.2.3 Posisi dan Relevansi Penelitian

Permasalahan *Colored Queens* merupakan topik yang relatif baru dalam literatur akademis. Meskipun permainan ini telah menarik perhatian luas sejak diperkenalkan oleh LinkedIn sebagai *puzzle* harian [6], kajian ilmiah formal terhadap permasalahan ini masih sangat terbatas. Sejauh tinjauan pustaka yang dilakukan, satu-satunya publikasi akademis yang secara langsung memformulasikan permainan LinkedIn Queens adalah karya Mata Ali dan Mencia [8], yang mengusulkan formulasi *Quadratic Unconstrained Binary Optimization* (QUBO) untuk menyelesaikan generalisasi permainan tersebut pada perangkat keras kuantum. Pendekatan-pendekatan lain yang tersedia di komunitas pengembang umumnya bersifat informal dan tidak dipublikasikan melalui kanal akademis, seperti

implementasi *backtracking* sederhana, formulasi *integer programming*, serta penggunaan *SAT/SMT solver*.

Celah penelitian (*research gap*) yang teridentifikasi dari tinjauan ini adalah sebagai berikut. Pertama, belum terdapat kajian yang memodelkan *Colored Queens* secara eksplisit sebagai *Constraint Satisfaction Problem* (CSP) dan menyelesaikannya menggunakan teknik-teknik klasik CSP seperti *backtracking* dengan *forward checking* dan *arc consistency*. Kedua, belum ditemukan penelitian yang membandingkan pendekatan sistematis (*complete*) dengan pendekatan *metaheuristic* untuk permasalahan ini. Ketiga, analisis empiris mengenai pengaruh ukuran papan terhadap performa algoritma penyelesaian juga belum dilakukan secara formal.

Tugas akhir ini bertujuan untuk mengisi celah tersebut dengan memberikan dua kontribusi utama. Kontribusi pertama adalah pemodelan *Colored Queens* sebagai CSP dan penyelesaiannya menggunakan *backtracking* yang diperkuat dengan *forward checking* dan *Arc Consistency* (AC-3). Kontribusi kedua adalah penerapan *Particle Swarm Optimization* (PSO) yang diadaptasi untuk ruang pencarian diskrit sebagai pendekatan alternatif. Perbandingan antara kedua pendekatan tersebut diharapkan memberikan wawasan mengenai *trade-off* antara kelengkapan solusi dan efisiensi waktu, serta perilaku masing-masing algoritma pada papan berukuran kecil hingga besar.

2.3 Constraint Satisfaction Problem

2.3.1 Definisi dan Komponen CSP

Constraint Satisfaction Problem (CSP) merupakan kerangka kerja formal untuk merepresentasikan dan menyelesaikan permasalahan yang melibatkan sekumpulan variabel dengan batasan-batasan yang harus dipenuhi secara bersamaan. Secara formal, sebuah CSP didefinisikan sebagai tripel $\mathcal{P} = \langle X, D, C \rangle$ [3] di mana:

- $X = \{x_1, x_2, \dots, x_n\}$ adalah himpunan variabel yang nilainya harus ditentukan.
- $D = \{D_1, D_2, \dots, D_n\}$ adalah himpunan domain, di mana D_i menyatakan himpunan nilai yang dapat diambil oleh variabel x_i .
- $C = \{C_1, C_2, \dots, C_m\}$ adalah himpunan kendala, di mana setiap kendala C_j mendefinisikan kombinasi nilai yang diperbolehkan untuk suatu subhimpunan variabel tertentu.

Sebuah *assignment* adalah pemetaan dari satu atau lebih variabel ke nilai-nilai dalam domainnya. *Assignment* disebut *consistent* apabila tidak melanggar kendala manapun, dan disebut *complete* apabila seluruh variabel telah diberi nilai. Solusi dari sebuah CSP adalah *complete assignment* yang bersifat *consistent*, yaitu seluruh variabel telah diberi nilai dan seluruh kendala terpenuhi [3].

Kajian formal mengenai CSP berawal dari karya Montanari pada tahun 1974 yang memperkenalkan kerangka jaringan kendala (*constraint network*), dan dilanjutkan oleh Mackworth pada tahun 1977 yang mengembangkan algoritma *arc consistency* sebagai teknik dasar untuk mereduksi ruang pencarian [9]. Sejak saat itu, CSP telah menjadi paradigma fundamental dalam *Artificial Intelligence* dan digunakan secara luas untuk memodelkan berbagai permasalahan seperti penjadwalan, pewarnaan graf, dan penyelesaian *puzzle* [3].

2.3.2 Teknik Representasi

Sebuah CSP dapat direpresentasikan secara visual menggunakan *constraint graph*, yaitu sebuah graf di mana setiap simpul merepresentasikan variabel dan setiap sisi menghubungkan dua variabel yang terlibat dalam suatu kendala bersama [3]. Representasi ini berguna untuk menganalisis struktur ketergantungan antarvariabel serta menentukan strategi penyelesaian yang tepat.

Kendala dalam CSP dapat diklasifikasikan berdasarkan jumlah variabel yang terlibat. Kendala *unary* melibatkan satu variabel dan membatasi domain variabel tersebut secara langsung, misalnya $x_1 \neq 3$. Kendala *binary* melibatkan dua variabel dan membatasi kombinasi nilai yang diperbolehkan di antara keduanya, misalnya $x_1 \neq x_2$. Dalam praktiknya, sebagian besar CSP dapat ditransformasikan ke dalam bentuk yang hanya mengandung kendala *binary* tanpa kehilangan informasi [3], dan banyak algoritma penyelesaian CSP dirancang secara khusus untuk menangani kendala jenis ini.

Struktur *constraint graph* memiliki implikasi langsung terhadap kompleksitas penyelesaian. Apabila graf memiliki struktur pohon (*tree-structured*), CSP dapat diselesaikan dalam waktu polinomial. Sebaliknya, graf yang padat (*dense*) dengan banyak sisi mengindikasikan interaksi kendala yang tinggi dan umumnya memerlukan teknik penyelesaian yang lebih canggih [3].

2.3.3 Pemodelan Colored Queens sebagai CSP

Permasalahan *Colored Queens* dapat dimodelkan secara natural sebagai CSP dengan mendefinisikan komponen-komponen $\langle X, D, C \rangle$ sebagai berikut.

Himpunan variabel $X = \{x_1, x_2, \dots, x_n\}$ merepresentasikan n buah sektor warna pada papan, di mana setiap variabel x_i bersesuaian dengan sektor warna ke- i . Nilai yang diberikan pada x_i menyatakan posisi sel tempat menteri ditempatkan dalam sektor tersebut.

Domain D_i untuk setiap variabel x_i adalah himpunan seluruh sel yang termasuk dalam sektor warna ke- i . Secara formal, apabila sektor warna ke- i terdiri dari sel-sel $\{(r_1, c_1), (r_2, c_2), \dots, (r_k, c_k)\}$, maka $D_i = \{(r_1, c_1), (r_2, c_2), \dots, (r_k, c_k)\}$. Perlu dicatat bahwa ukuran domain bervariasi antarsektor karena bentuk sektor yang ireguler, sehingga $|D_i| \neq |D_j|$ pada umumnya.

Himpunan kendala C terdiri dari empat jenis kendala yang bersesuaian dengan aturan permainan:

1. **Kendala baris:** untuk setiap pasangan variabel x_i, x_j dengan $i \neq j$, apabila $x_i = (r_i, c_i)$ dan $x_j = (r_j, c_j)$, maka $r_i \neq r_j$.
2. **Kendala kolom:** untuk setiap pasangan variabel x_i, x_j dengan $i \neq j$, berlaku $c_i \neq c_j$.
3. **Kendala sektor warna:** dipenuhi secara implisit oleh definisi variabel, karena setiap variabel hanya dapat mengambil nilai dari sektornya sendiri.
4. **Kendala adjacency:** untuk setiap pasangan variabel x_i, x_j dengan $i \neq j$, apabila $x_i = (r_i, c_i)$ dan $x_j = (r_j, c_j)$, maka $|r_i - r_j| > 1$ atau $|c_i - c_j| > 1$.

Seluruh kendala di atas bersifat *binary*, yaitu melibatkan tepat dua variabel, sehingga permasalahan *Colored Queens* menghasilkan *constraint graph* yang bersifat *complete*—setiap pasangan variabel terhubung oleh setidaknya satu kendala. Karakteristik ini mengindikasikan tingkat interaksi kendala yang tinggi dan memperkuat argumen bahwa teknik propagasi kendala diperlukan untuk mereduksi ruang pencarian secara efektif sebelum atau selama proses pencarian solusi. Teknik-teknik penyelesaian tersebut akan dibahas pada bagian berikutnya.

2.4 Teknik Penyelesaian CSP

Bagian ini membahas teknik-teknik yang digunakan untuk menyelesaikan CSP secara umum, yang kemudian akan diterapkan pada permasalahan *Colored Queens*. Teknik-teknik tersebut dikelompokkan menjadi dua kategori besar: algoritma sistematis yang menjamin kelengkapan pencarian, dan pendekatan *metaheuristic* yang bersifat probabilistik namun mampu menjelajahi ruang solusi yang sangat besar secara efisien.

2.4.1 Algoritma Sistematis

Algoritma sistematis untuk CSP adalah algoritma yang menjamin ditemukannya solusi apabila solusi tersebut ada, atau membuktikan ketiadaan solusi apabila tidak ada satu pun penugasan yang memenuhi seluruh kendala. Jaminan kelengkapan (*completeness*) ini membedakan algoritma sistematis dari pendekatan *metaheuristic* yang akan dibahas pada Bagian 2.4.2. Dalam konteks CSP, keluarga algoritma sistematis yang paling banyak digunakan dibangun di atas fondasi *backtracking search*, yang kemudian diperkuat oleh teknik-teknik propagasi kendala seperti *forward checking* dan *arc consistency* [3].

Backtracking Search

Backtracking search adalah algoritma pencarian berbasis *depth-first search* (DFS) yang membangun solusi CSP secara inkremental, satu variabel dalam satu langkah [3]. Pada setiap langkah, algoritma memilih sebuah variabel yang belum diberi nilai, lalu mencoba setiap nilai dalam domain variabel tersebut secara berurutan. Apabila sebuah nilai tidak melanggar kendala apapun terhadap variabel-variabel yang telah ditetapkan sebelumnya, nilai tersebut ditetapkan dan algoritma melanjutkan ke variabel berikutnya secara rekursif. Apabila tidak ada nilai yang konsisten ditemukan untuk variabel saat ini, algoritma mundur (*backtrack*) ke variabel sebelumnya dan mencoba nilai lain dari domain variabel tersebut.

Pseudocode dasar *backtracking search* diperlihatkan pada Listing 2.1.

Kode 2.1: Pseudocode Backtracking Search untuk CSP

```

25 function BACKTRACK(assignment, csp):
26   if assignment is complete:
27     return assignment
28   var = SELECT_UNASSIGNED_VARIABLE(csp)
29   for each value in ORDER_DOMAIN_VALUES(var, assignment, csp):
30     if value is consistent with assignment:
31       add {var = value} to assignment
32       result = BACKTRACK(assignment, csp)
33       if result != failure:
34         return result
35       remove {var = value} from assignment
36   return failure

```

Keunggulan utama *backtracking search* dibandingkan pencarian *brute-force* naif adalah pemangkasan ruang pencarian (*pruning*) yang terjadi secara otomatis: setiap kali penugasan parsial terdeteksi melanggar kendala, seluruh cabang pencarian di bawahnya dieliminasi tanpa perlu dieksplorasi. Secara formal, kompleksitas waktu terburuk dari *backtracking* pada CSP dengan n variabel dan ukuran domain maksimum d adalah $O(d^n)$ [10]. Meskipun tetap eksponensial, angka ini jauh lebih kecil dibandingkan pencarian lengkap yang mengevaluasi seluruh d^n kombinasi tanpa pemangkasan.

Performa *backtracking search* sangat dipengaruhi oleh dua keputusan heuristik: urutan pemilihan variabel dan urutan pemilihan nilai dalam domain. Heuristik *Minimum Remaining Values* (MRV) memilih variabel yang memiliki jumlah nilai valid terkecil dalam domainnya, berdasarkan intuisi bahwa variabel yang paling terbatas cenderung menyebabkan kegagalan lebih awal sehingga pemangkasan terjadi di level pohon pencarian yang lebih tinggi [3]. Di sisi lain, heuristik *Least Constraining Value* (LCV) memilih nilai yang paling sedikit membatasi domain variabel-variabel lainnya, dengan tujuan memaksimalkan fleksibilitas untuk langkah-langkah selanjutnya.

Meskipun heuristik-heuristik tersebut dapat mengurangi jumlah *backtrack* secara signifikan, *backtracking* murni masih rentan terhadap fenomena *thrashing*, yaitu pengulangan kegagalan yang sama berulang kali akibat konflik yang tidak terdeteksi secara dini [10]. Sebagai contoh, apabila dua variabel yang belum ditugaskan memiliki domain yang saling berkonflik, *backtracking* murni baru akan mendeteksi konflik tersebut setelah salah satu dari keduanya diproses, yang mungkin baru terjadi setelah banyak langkah rekursi. Kelemahan inilah yang mendorong pengembangan teknik propagasi kendala yang diintegrasikan ke dalam proses *backtracking*, dimulai dari *forward checking* pada bagian berikutnya.

Forward Checking

Forward checking adalah teknik propagasi kendala yang diintegrasikan ke dalam *backtracking search* untuk mendeteksi kegagalan lebih awal [11]. Ide dasarnya adalah: setiap kali sebuah variabel x_i diberi nilai v , algoritma segera memeriksa seluruh variabel yang belum diberi nilai dan terhubung dengan x_i melalui suatu kendala, lalu menghapus dari domain variabel-variabel tersebut semua nilai yang tidak konsisten dengan penugasan $x_i = v$. Proses ini dilakukan secara langsung pada saat penugasan, bukan menunggu hingga variabel-variabel tersebut giliran diproses.

Pseudocode *forward checking* diperlihatkan pada Listing 2.2.

Kode 2.2: Pseudocode Forward Checking

```

24
25.1 function FORWARD.CHECK(var, value, assignment, csp):
26.2   for each unassigned variable Y constrained with var:
27.3     for each v in Domain(Y):
28.4       if v is inconsistent with value given constraint(var, Y):
29.5         remove v from Domain(Y)
30.6       if Domain(Y) is empty:
31.7         return failure
32.8     return success

```

Keuntungan langsung dari *forward checking* adalah kemampuannya untuk mendeteksi *domain wipeout* — yaitu kondisi di mana domain suatu variabel yang belum ditugaskan menjadi kosong — jauh sebelum variabel tersebut giliran diproses oleh rekursi. Apabila *domain wipeout* terdeteksi, algoritma dapat segera melakukan *backtrack* tanpa perlu mengeksplorasi cabang-cabang yang dipastikan tidak produktif. Dengan demikian, *forward checking* mengurangi kedalaman pohon pencarian yang perlu dijelajahi sebelum kegagalan ditemukan.

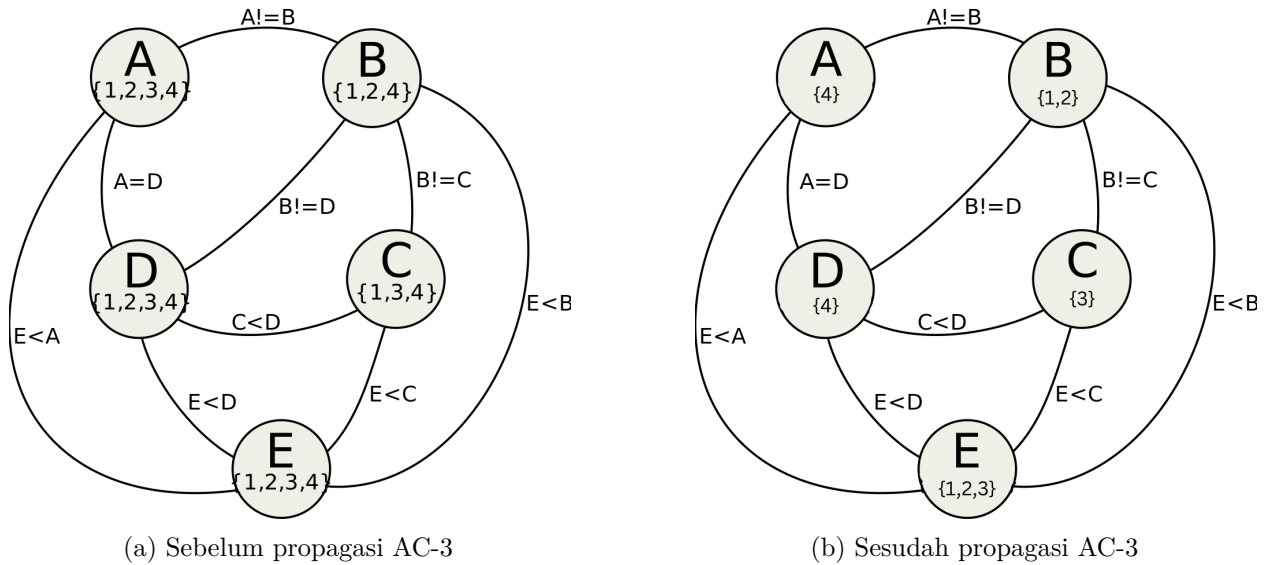
Dalam konteks *Colored Queens*, *forward checking* bekerja sebagai berikut: ketika sebuah menteri ditempatkan pada posisi (r, c) di sektor warna tertentu, algoritma segera memindai domain seluruh sektor warna yang belum diproses dan menghapus setiap posisi yang berada pada baris r , kolom c , atau sel-sel yang bertetangga (*adjacent*) dengan (r, c) . Mengingat *constraint graph* pada *Colored Queens* bersifat *complete* sebagaimana dibahas pada Bagian 2.3.3, setiap penugasan memengaruhi

domain seluruh variabel lainnya, sehingga *forward checking* memberikan dampak reduksi yang substansial pada setiap langkah.

Namun, *forward checking* memiliki keterbatasan fundamental: teknik ini hanya memeriksa konsistensi antara variabel yang baru ditugaskan dengan variabel-variabel yang belum ditugaskan, tanpa memeriksa konsistensi antarsesama variabel yang belum ditugaskan [3]. Sebagai ilustrasi, misalkan setelah *forward checking*, domain sektor warna A hanya menyisakan satu posisi di baris 3, sementara domain sektor warna B juga hanya menyisakan satu posisi di baris 3. Kedua domain ini saling bertentangan karena melanggar kendala baris, namun *forward checking* tidak akan mendeteksi konflik tersebut karena keduanya masih merupakan variabel yang belum ditugaskan. Konflik baru akan ditemukan ketika salah satu dari kedua variabel tersebut diproses, yang berpotensi membuang banyak langkah pencarian. Keterbatasan inilah yang diatasi oleh teknik *arc consistency* yang dibahas pada bagian berikutnya.

Arc Consistency (AC-3)

Arc consistency adalah properti konsistensi yang lebih kuat dibandingkan yang ditegakkan oleh *forward checking*. Sebuah *arc* (x_i, x_j) dalam CSP dikatakan *arc-consistent* apabila untuk setiap nilai $v \in D_i$, terdapat setidaknya satu nilai $w \in D_j$ sedemikian sehingga penugasan $x_i = v$ dan $x_j = w$ memenuhi semua kendala di antara x_i dan x_j [12]. Nilai w tersebut disebut sebagai *support* untuk v pada *arc* (x_i, x_j) . Apabila suatu nilai $v \in D_i$ tidak memiliki *support* pada D_j , maka v dapat dihapus dari D_i tanpa menghilangkan solusi valid manapun, karena v dipastikan tidak dapat menjadi bagian dari solusi selama domain x_j tetap seperti saat ini.



Gambar 2.4: Contoh propagasi AC-3 pada *constraint graph* dengan lima variabel. Sebelum propagasi (a), domain variabel masih lebar. Setelah AC-3 diterapkan (b), nilai-nilai yang tidak memiliki *support* dieliminasi, menghasilkan domain yang jauh lebih kecil.

Sebuah CSP dikatakan *arc-consistent* secara keseluruhan apabila seluruh *arc* di dalamnya bersifat *arc-consistent*. Perbedaan utama dengan *forward checking* terletak pada cakupan pemeriksaan: *forward checking* hanya memeriksa konsistensi antara variabel yang baru ditugaskan dengan variabel

lain, sedangkan *arc consistency* memeriksa konsistensi antara *seluruh* pasangan variabel, termasuk antarsesama variabel yang belum ditugaskan [3].

Algoritma AC-3 (*Arc Consistency Algorithm #3*), yang diperkenalkan oleh Mackworth [12], merupakan algoritma standar untuk menegakkan *arc consistency*. AC-3 mengelola sebuah antrean Q yang berisi seluruh *arc* yang perlu diperiksa konsistensinya. Untuk setiap *arc* (x_i, x_j) yang diambil dari antrean, fungsi REVISE memeriksa apakah ada nilai dalam D_i yang tidak memiliki *support* di D_j ; nilai-nilai tanpa *support* dihapus dari D_i . Apabila D_i berubah (ada penghapusan), maka seluruh *arc* (x_k, x_i) untuk setiap tetangga x_k dari x_i (selain x_j) ditambahkan kembali ke antrean, karena penghapusan nilai dari D_i berpotensi membuat beberapa nilai di D_k kehilangan *support*-nya. Proses berlanjut hingga antrean kosong (*arc consistency* tercapai) atau domain suatu variabel menjadi kosong (inkonsistensi terdeteksi). Pseudocode AC-3 diperlihatkan pada Listing 2.3.

Kode 2.3: Pseudocode Algoritma AC-3

```

12  function AC3(csp):
13  Q = {all arcs (Xi, Xj) in csp}
14  while Q is not empty:
15      (Xi, Xj) = REMOVE_FROM_QUEUE(Q)
16      if REVISE(csp, Xi, Xj):
17          if Domain(Xi) is empty:
18              return false
19      for each Xk that is a neighbor of Xi, Xk != Xj:
20          add (Xk, Xi) to Q
21  return true
22
23  function REVISE(csp, Xi, Xj):
24  revised = false
25  for each v in Domain(Xi):
26      if no value w in Domain(Xj) satisfies constraint(Xi, Xj):
27          remove v from Domain(Xi)
28          revised = true
29  return revised
30
31

```

Kompleksitas waktu AC-3 adalah $O(cd^3)$, di mana c adalah jumlah kendala (*arc*) dan d adalah ukuran maksimum domain [12]. Analisisnya adalah sebagai berikut: setiap *arc* dapat dimasukkan ke dalam antrean paling banyak d kali (karena setiap kali *arc* diproses ulang, setidaknya satu nilai telah dihapus dari domain terkait), dan setiap pemanggilan fungsi REVISE memerlukan waktu $O(d^2)$ untuk memeriksa setiap pasangan nilai. Meskipun terdapat algoritma yang lebih efisien seperti AC-4 dengan kompleksitas optimal $O(cd^2)$ [13], AC-3 tetap menjadi pilihan yang paling banyak digunakan dalam praktik karena kesederhanaan implementasinya dan *overhead* per-operasi yang rendah.

Dalam konteks penyelesaian CSP berbasis *backtracking*, AC-3 digunakan sebagai teknik propagasi yang dijalankan setelah *forward checking* pada setiap langkah penugasan. Pendekatan gabungan ini, yang sering disebut *Maintaining Arc Consistency* (MAC), bekerja sebagai berikut [14]: setelah sebuah variabel diberi nilai dan *forward checking* menghapus nilai-nilai yang secara langsung tidak konsisten dari domain variabel-variabel yang belum ditugaskan, AC-3 dijalankan pada seluruh *arc* di antara variabel-variabel yang belum ditugaskan untuk mendeteksi dan mengeliminasi inkonsistensi transitif yang tidak terdeteksi oleh *forward checking*. Apabila proses propagasi ini menghasilkan *domain wipeout* pada salah satu variabel, *backtrack* dilakukan segera. Kombinasi *backtracking* + *forward checking* + AC-3 inilah yang menjadi fondasi solver sistematis pada tugas akhir ini dan implementasinya akan dibahas secara rinci pada Bab ??.

2.4.2 Metaheuristic

Konsep Umum Metaheuristic

Metaheuristic adalah kerangka kerja algoritma tingkat tinggi yang dirancang untuk memandu proses pencarian solusi pada masalah optimasi yang ruang pencariannya terlalu besar atau terlalu kompleks untuk dijelajahi secara sistematis [15]. Istilah ini berasal dari gabungan kata Yunani *meta* (“di atas”) dan *heuriskein* (“menemukan”), yang secara harfiah berarti “strategi pencarian tingkat tinggi.” Berbeda dengan algoritma sistematis yang dibahas pada Bagian 2.4.1, *metaheuristic* bersifat *incomplete*: tidak ada jaminan bahwa solusi optimal — atau bahkan solusi valid sekalipun — akan selalu ditemukan. Sebagai gantinya, *metaheuristic* menawarkan kemampuan untuk menavigasi ruang solusi yang sangat besar secara efisien, dengan harapan menemukan solusi yang cukup baik dalam waktu yang jauh lebih singkat dibandingkan pencarian eksak.

Karakteristik utama yang membedakan *metaheuristic* dari pencarian lokal sederhana (*simple local search*) adalah adanya mekanisme untuk menghindari jebakan *local optimum* [15]. Pencarian lokal sederhana, seperti *hill climbing*, rentan terjebak pada solusi yang optimal secara lokal tetapi bukan optimal secara global, karena algoritma berhenti begitu tidak ada tetangga yang lebih baik dari solusi saat ini. *Metaheuristic* mengatasi keterbatasan ini melalui berbagai mekanisme, antara lain penerimaan solusi yang sementara lebih buruk (seperti pada *Simulated Annealing*), penggunaan memori terhadap solusi-solusi yang telah dikunjungi (seperti pada *Tabu Search*), atau pemanfaatan populasi solusi yang berinteraksi satu sama lain (seperti pada *Genetic Algorithm* dan *Particle Swarm Optimization*).

Secara umum, *metaheuristic* dapat diklasifikasikan berdasarkan beberapa dimensi [15]:

1. **Berbasis trajektori vs berbasis populasi.** Algoritma berbasis trajektori, seperti *Simulated Annealing* dan *Tabu Search*, bekerja dengan satu solusi yang dimodifikasi secara iteratif. Algoritma berbasis populasi, seperti *Genetic Algorithm* (GA) dan *Particle Swarm Optimization* (PSO), bekerja dengan sekumpulan solusi yang berevolusi secara paralel. Pendekatan berbasis populasi memiliki keunggulan natural dalam hal eksplorasi karena memelihara diversitas solusi di dalam populasi.
2. **Eksplorasi vs eksploitasi.** Setiap *metaheuristic* harus menyeimbangkan dua kecenderungan yang saling berlawanan: *eksplorasi* (menjelajahi area baru dalam ruang solusi) dan *eksploitasi* (mengintensifkan pencarian di sekitar solusi-solusi terbaik yang telah ditemukan). Terlalu banyak eksplorasi menyebabkan pencarian menjadi acak dan tidak efisien, sementara terlalu banyak eksploitasi menyebabkan konvergensi prematur ke *local optimum*.
3. **Terinspirasi alam vs non-terinspirasi alam.** Banyak *metaheuristic* terinspirasi dari fenomena biologis atau fisika, seperti evolusi genetik (GA), perilaku kawanan (PSO), dan pendinginan logam (*Simulated Annealing*). Meskipun inspirasi alam memberikan intuisi yang berguna, efektivitas algoritma pada akhirnya bergantung pada mekanisme komputasional yang mendasarinya, bukan pada kesetiaan terhadap analogi biologis.

Dalam konteks penyelesaian *Constraint Satisfaction Problem* (CSP), penggunaan *metaheuristic* memerlukan reformulasi masalah dari paradigma “cari penugasan yang memenuhi seluruh kendala” menjadi paradigma “minimisasi jumlah pelanggaran kendala” [16]. Dengan kata lain, CSP ditransformasikan menjadi *Constraint Optimization Problem* (COP) di mana fungsi objektif (*fitness*

1 *function*) mengukur seberapa banyak kendala yang dilanggar oleh suatu solusi kandidat. Solusi valid
2 dalam CSP asli berkorespondensi dengan solusi yang memiliki nilai *fitness* nol — yaitu, tidak ada
3 kendala yang dilanggar. Reformulasi ini memungkinkan penerapan *metaheuristic* yang pada awalnya
4 dirancang untuk masalah optimasi kontinu ke permasalahan kombinatorik diskrit seperti *Colored*
5 *Queens*, meskipun diperlukan adaptasi representasi solusi dan operator pencarian sebagaimana akan
6 dibahas pada bagian berikutnya.

7 Representasi Solusi Diskrit

8 Particle Swarm Optimization

9 2.5 Perbandingan Pendekatan

10 2.5.1 Complete vs Metaheuristic

11 2.5.2 Backtracking+FC+AC-3 vs PSO

12 2.5.3 Justifikasi Pemilihan Metode

DAFTAR REFERENSI

- [1] Bell, J. dan Stevens, B. (2009) A survey of known results and research areas for n-queens. *Discrete mathematics*, **309**, 1–31. <https://www.sciencedirect.com/science/article/pii/S0012365X07010394#b20> Diakses: 2026-05-10.
- [2] Glock, S., Munhá Correia, D., dan Sudakov, B. (2022) The n-queens completion problem. *Research in the Mathematical Sciences*, **9**, 41. <https://link.springer.com/article/10.1007/s40687-022-00335-1> Diakses: 2026-05-10.
- [3] Russell, S. J. (2010) *Artificial intelligence a modern approach*. Pearson Education, Inc., New Jersey. , Chapter 6 [http://repo.darmajaya.ac.id/5272/1/Artificial%20Intelligence-A%20Modern%20Approach%20\(3rd%20Edition\)%20\(%20PDFDrive%20\).pdf](http://repo.darmajaya.ac.id/5272/1/Artificial%20Intelligence-A%20Modern%20Approach%20(3rd%20Edition)%20(%20PDFDrive%20).pdf) Diakses: 2026-05-10.
- [4] Hoffman, E. J., Loessi, J., dan Moore, R. C. (1969) Constructions for the solution of the m queens problem. *Mathematics Magazine*, **42**, 66–72. <https://exp-blog.com/algorithm/poj/poj3239-solution-to-the-n-queens-puzzle/doc/Constructions%20for%20the%20Solution%20of%20the%20m%20Queens%20Problem.pdf> Diakses: 2026-05-14.
- [5] Gent, I. P., Jefferson, C., dan Nightingale, P. (2017) Complexity of n-queens completion. *Journal of Artificial Intelligence Research*, **59**, 815–848. <https://www.jair.org/index.php/jair/article/view/11079/26262> Diakses: 2026-05-14.
- [6] LinkedIn (2024) Play queens game on linkedin. <https://www.linkedin.com/help/linkedin/answer/a6269510>. Diakses: 2026-05-14.
- [7] Snyder, T. (2013) Star battle rules and info. <https://www.gmpuzzles.com/blog/star-battle-rules-and-info/>. Diakses: 14 Mei 2026.
- [8] Ali, A. M. dan Mencia, E. (2026) A qubo formulation for the generalized linkedin queens and takuzu/tango game.
- [9] Montanari, U. (1974) Networks of constraints: Fundamental properties and applications to picture processing. *Information sciences*, **7**, 95–132. <http://cse.unl.edu/~choueiry/Documents/Montanari-74.pdf> Diakses 14 Mei 2026.
- [10] Kumar, V. (1998) Algorithms for constraint satisfaction problems: A survey. *A.I. Mag*, **13**.
- [11] Van Beek, P. (2006) Backtracking search algorithms. *Handbook of Constraint Programming*, pp. 85–134. Elsevier. <https://cs.uwaterloo.ca/~vanbeek/Publications/survey06.pdf> Diakses: 17 Mei 2026.
- [12] Mackworth, A. K. (1977) Consistency in networks of relations. *Artificial intelligence*, **8**, 99–118. <https://www.cs.ubc.ca/~mack/Publications/AI77.pdf> Diakses: 17 Mei 2026.
- [13] Mohr, R. dan Henderson, T. C. (1986) Arc and path consistency revisited. *Artificial intelligence*, **28**, 225–233. <https://sci-hub.mk/10.1016/0004-3702%2886%2990083-4> Diakses: 17 Mei 2026.

- [14] Bessiere, C. (2006) Constraint propagation. *Handbook of Constraint Programming*, pp. 29–83. Elsevier. https://www.dcs.gla.ac.uk/~pat/cpM/papers/CP_Handbook-20060315-final.pdf Diakses: 17 Mei 2026.
- [15] Blum, C. dan Roli, A. (2003) Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM computing surveys (CSUR)*, **35**, 268–308. <https://dl.acm.org/doi/epdf/10.1145/937503.937505> Diakses: 17 Mei 2026.
- [16] Hoos, H. H. dan Tsang, E. (2006) Local search methods. *Handbook of Constraint Programming*, pp. 245–268. Elsevier. https://www.dcs.gla.ac.uk/~pat/cpM/papers/CP_Handbook-20060315-final.pdf Diakses: 17 Mei 2026.

LAMPIRAN A

KODE PROGRAM

Kode A.1: MyCode.c

```
1 // This does not make algorithmic sense,
2 // but it shows off significant programming characters.
3
4 #include<stdio.h>
5
6 void myFunction( int input, float* output ) {
7     switch ( array[i] ) {
8         case 1: // This is silly code
9             if ( a >= 0 || b <= 3 && c != x )
10                 *output += 0.005 + 20050;
11             char = 'g';
12             b = 2^n + ~right_size - leftSize * MAX_SIZE;
13             c = (--aaa + &daa) / (bbb++ - ccc % 2 );
14             strcpy(a,"hello_$@?");
15         }
16         count = ~mask | 0x00FF00AA;
17     }
18 }
19
20 // Fonts for Displaying Program Code in LATEX
21 // Adrian P. Robson, nepsweb.co.uk
22 // 8 October 2012
23 // http://nepsweb.co.uk/docs/progfonts.pdf
```

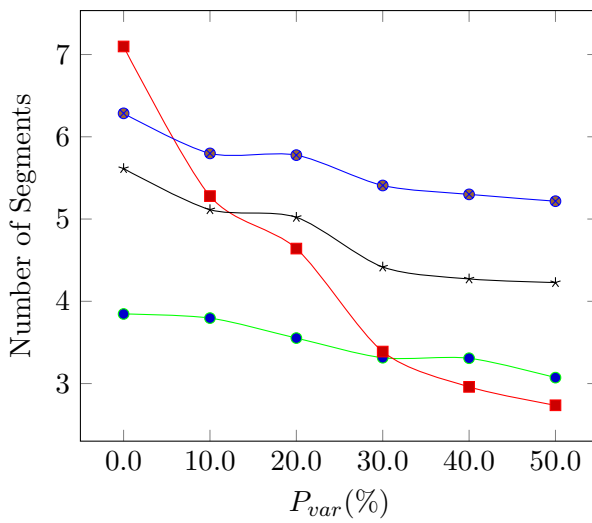
Kode A.2: MyCode.java

```
1 import java.util.ArrayList;
2 import java.util.Collections;
3 import java.util.HashSet;
4
5 //class for set of vertices close to furthest edge
6 public class MyFurSet {
7     protected int id; //id of the set
8     protected MyEdge FurthestEdge; //the furthest edge
9     protected HashSet<MyVertex> set; //set of vertices close to furthest edge
10    protected ArrayList<ArrayList<Integer>> ordered; //list of all vertices in the set for each trajectory
11    protected ArrayList<Integer> closeID; //store the ID of all vertices
12    protected ArrayList<Double> closeDist; //store the distance of all vertices
13    protected int totaltrj; //total trajectories in the set
14
15    /*
16     * Constructor
17     * @param id : id of the set
18     * @param totaltrj : total number of trajectories in the set
19     * @param FurthestEdge : the furthest edge
20     */
21    public MyFurSet(int id,int totaltrj,MyEdge FurthestEdge) {
22        this.id = id;
23        this.totaltrj = totaltrj;
24        this.FurthestEdge = FurthestEdge;
25        set = new HashSet<MyVertex>();
26        ordered = new ArrayList<ArrayList<Integer>>();
27        for (int i=0;i<totaltrj;i++) ordered.add(new ArrayList<Integer>());
28        closeID = new ArrayList<Integer>(totaltrj);
29        closeDist = new ArrayList<Double>(totaltrj);
30        for (int i = 0;i <totaltrj;i++) {
31            closeID.add(-1);
32            closeDist.add(Double.MAX_VALUE);
33        }
34    }
35
36 }
```

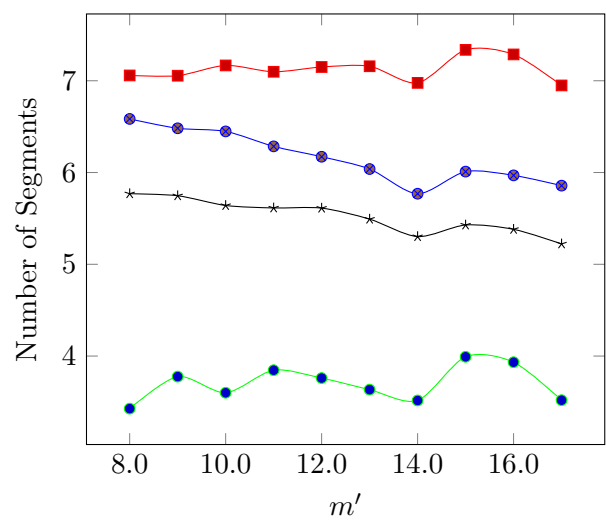

LAMPIRAN B

HASIL EKSPERIMEN

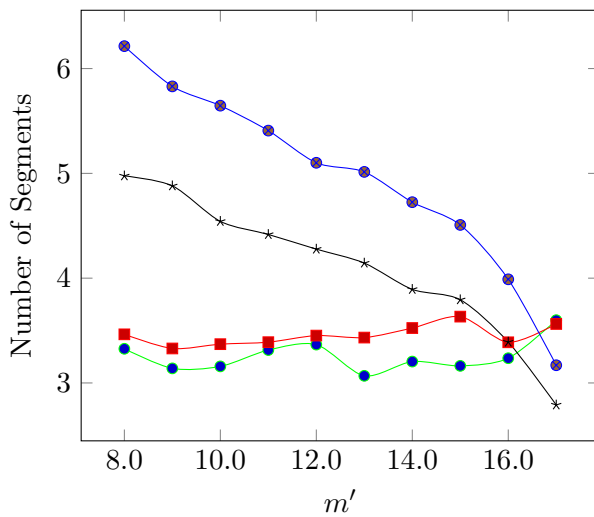
Hasil eksperimen berikut dibuat dengan menggunakan TIKZPICTURE (bukan hasil excel yg diubah ke file bitmap). Sangat berguna jika ingin menampilkan tabel (yang kuantitasnya sangat banyak) yang datanya dihasilkan dari program komputer.



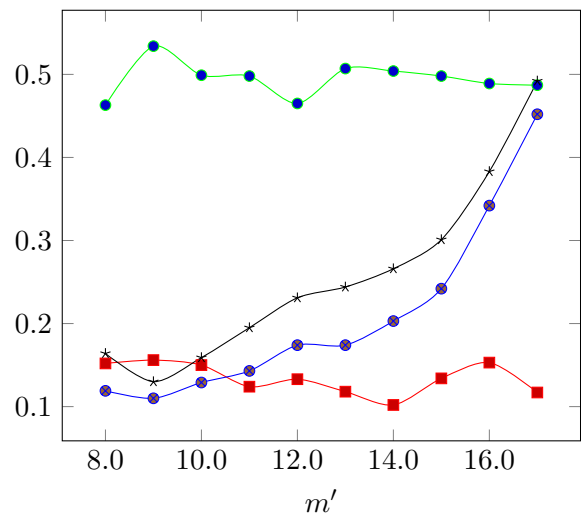
Gambar B.1: Hasil 1



Gambar B.2: Hasil 2



Gambar B.3: Hasil 3



Gambar B.4: Hasil 4